

Novelty-aware Graph Traversal and Expansion for Hierarchical Reinforcement Learning

Jongchan Park*

Sungkyunkwan University
Department of Artificial Intelligence
Suwon, Gyeonggi-do, South Korea
marin0625@naver.com

Seungjun Oh*

Sungkyunkwan University
Department of Artificial Intelligence
Suwon, Gyeonggi-do, South Korea
seungjun1614@gmail.com

Yusung Kim†

Sungkyunkwan University
Department of Artificial Intelligence
Suwon, Gyeonggi-do, South Korea
yskim525@skku.edu

Abstract

Hierarchical Reinforcement Learning (HRL) is specially designed for environments characterized by long-term goals and sparse rewards. High-level policies in HRL learn to generate appropriate subgoals aimed at accomplishing the final goal, while low-level policies focus on achieving these designated subgoals. Recently, graph-based HRL algorithms have demonstrated enhanced learning capabilities through the structural representation of state spaces as graphs. However, existing graph-based HRL methods still often generate inefficient subgoals. This paper introduces a new method, Novelty-aware Graph Traversal and Expansion (NGTE), which selects an optimal node at the graph boundary, termed an Outpost Subgoal, as a direct path toward the final goal. Once the Outpost Subgoal is reached, NGTE transitions into an exploration phase, offering exploration subgoals within a reachable distance to efficiently expand the graph. Demonstrated in complex environments such as quadruped robot navigation and robotic arm manipulation, NGTE consistently outperforms existing graph and non-graph HRL methods, showing outstanding performance, especially in the most challenging scenarios with fixed start and fixed goal conditions. The source code is available on <https://github.com/ihatebroccoli/NGTE.git>.

CCS Concepts

• Computing methodologies → Reinforcement learning.

Keywords

Hierarchical Reinforcement learning; Novelty Critic; Graph Planning

ACM Reference Format:

Jongchan Park, Seungjun Oh, and Yusung Kim. 2024. Novelty-aware Graph Traversal and Expansion for Hierarchical Reinforcement Learning. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*, October 21–25, 2024, Boise, ID, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3627673.3679523>

*Equal contribution

†Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '24, October 21–25, 2024, Boise, ID, USA.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0436-9/24/10

<https://doi.org/10.1145/3627673.3679523>

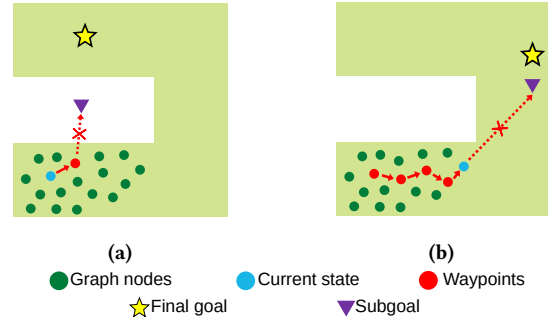


Figure 1: Previous graph-based HRL approaches that use graph path-finding, suffer from (a) generating in incorrect directions and (b) generating subgoals too far away, leading to inefficient interactions. These problems occur when high-level agents are insufficiently trained.

1 Introduction

Reinforcement learning (RL) has achieved remarkable successes across diverse domains, ranging from games [14, 24] to robotics [12, 29], chip placements [19], and even nuclear fusion plasma control [6]. The concept of hierarchical reinforcement learning (HRL) has emerged to tackle larger and more difficult challenges. HRL strategically breaks down complex problems into more manageable sub-problems, enhancing the effectiveness of the learning process [18, 22, 27]. Graph-based HRL algorithms have exhibited more performance improvements in recent times [15, 17, 30]. They achieve this by discretizing the state space using graph representations, consequently leading to a reduction in the agent’s state and action space. Particularly notable is the ability to leverage graph nodes, ensuring that lower-level policies reliably and efficiently achieve subgoals generated by higher-level policies.

However, in larger and sparse-reward environments, graph-based HRL also encounters challenges in learning. This paper highlights three primary issues. The first issue involves the high-level policy misunderstanding of the distance estimation to the final goal (unvisited yet) as illustrated in Figure 1(a), leading to the generation of subgoals in incorrect directions. The second issue relates to the generation of subgoals placed excessively far apart, depicted in Figure 1(b), causing the low-level policy to fail to reach the subgoal after it goes beyond the boundaries of the graph. The third issue is the challenge of achieving a large-scale state space due to the lack of sufficient high-level exploration effort.

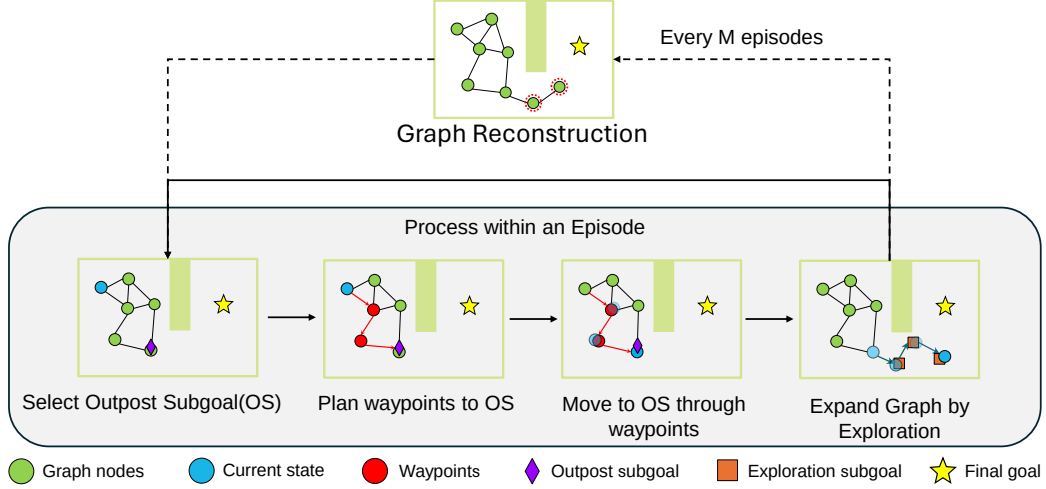


Figure 2: This is an overview of NGTE. (1) Graph construction: it samples uniform random states from Replay Buffer and builds a graph using a farthest point sampling (FPS) algorithm based on the distance prediction by Distance Critic. (2) **Select Outpost Subgoal:** by considering both Distance Critic and Novelty Critic, NGTE selects an outpost subgoal. (3) **Plan waypoints to OS:** find the shortest path to final goal With dijkstra algorithm. (4) **Move to OS through way:** The low-level policy iteratively reaches each waypoint until the outpost subgoal. (5) **Expand Graph by Exploration:** after reaching the outpost subgoal, NGTE transitions to an exploration phase to discover new states to expand the graph.

To address these challenges, we propose the Novelty-aware Graph Traversal and Expansion (NGTE) method. NGTE suggests the subgoal selection from graph nodes, in contrast to learning subgoal generations. This selection process considers not only the distance prediction from graph nodes to the final goal but also the expected novelty value up to the final goal. This enables the selection of an outpost subgoal located on the graph’s boundary towards the final goal, as a subgoal. Furthermore, choosing a graph node as a subgoal ensures that the low-level policy reaches the subgoal reliably. Upon reaching the subgoal at the graph’s boundary, NGTE seamlessly transitions into a constrained high-level exploration mode, accelerating graph expansion. Our simple subgoal selection and exploration strategy demonstrates the potential for achieving highly efficient HRL without the need for designing a sophisticated high-level policy.

Through comprehensive experiments in various quadruped robot navigation and robot arm manipulation environments, NGTE has shown its capability in selecting subgoals effectively and in facilitating rapid graph expansion. As a result, NGTE can significantly outperform previous HRL methods. Notably, NGTE has also successfully addressed the most challenging problem of achieving outstanding performance in fixed start and fixed goal conditions for the first time.

2 Related Work

2.1 Hierarchical RL

HRL has emerged to solve sparse and large problems. With decomposing long horizon tasks to sub-problems effectively, higher level can solve problem as wide sight and benefit from temporal abstraction [3, 16, 21]. Previous works primarily focused on alleviating

training inefficiency resulting from the non-stationary low-level policy during training. Namely, changing behavior of the low-level policy results in different transitions from the same high-level policy action. HIRO [20] proposed an off-policy correction technique that changes the high-level action to a subgoal which would cause the most similar sequence of transitions from the current low-level. HAC [18] relabels the original subgoal with the achieved goal, and impose a penalty reward on high-level actions that fail to reach with the low-level policy. Although these methods improve learning stability, unreliable subgoals generated from high-level policy cause limitation on efficient learning. HRAC [31] restricts the generated subgoal into the adjacent region via discretized adjacency matrix, and SAGA [28] enforces the high-level policy to generate feasible subgoals by updating the policy through adversarial loss. However, unreliable network outputs from less frequently visited states can lead to misdirected subgoal generation. Our proposed method NGTE has key differences from the line of these previous works. First, we select an outpost subgoal via Distance and Novelty Critic among the constructed graph nodes instead of a subgoal generating policy, thus does not suffer from the non-stationary issue resulting from a high-level policy. Furthermore, our outpost subgoal is ensured to reach naturally, since it is chosen from within the graph representing the visited states.

2.2 Graph-based HRL

Graph-based hierarchical reinforcement learning has received attention as a promising solution complex and sparse-reward problems [9, 13, 15, 30]. These approaches construct graphs and conduct planning over the graph by using graph search algorithms to decompose the complex tasks into easier goal-reaching problems.

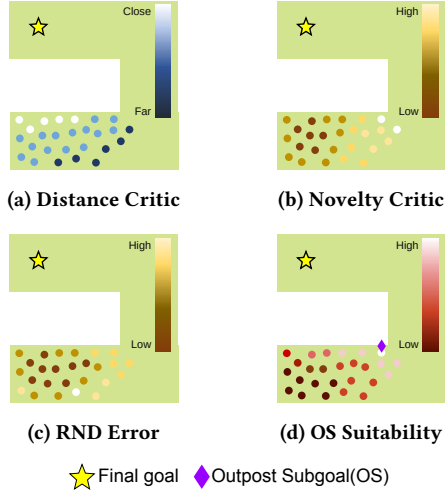


Figure 3: This figure shows an overview of subgoal selection in NGTE. The node colors correspond to the actual simulation results. (a) shows the distance prediction values by Distance Critic from each node to the final goal. (b) is the expected novelty values by our Novelty Critic from each node to the final goal. (c) RND Errors per node and they are used to train our Novelty Critic. (d) shows how NGTE selects an outpost subgoal by considering both Distance Critic and Novelty Critic values together.

However, these methods exhibit limitations to larger environments due to the fundamental challenge of HRL where the horizons of low and high-level policies are coupled. For example, if the intervals between high-level actions increase, the low-level policy becomes more challenging, and conversely, reducing the intervals decreases the learning efficiency of the high-level policy. DHRL [17] tackles this problem by increasing the subgoal generation interval in the high-level policy and employing waypoints on the graph for the low-level policy to sequentially reach the waypoint during a shorter interval. Our method is related to previous graph-based HRL approaches in the aspect of utilizing waypoints toward the subgoal and decoupling the horizon of both levels of policies. However, our method, NGTE, differs from them in the following ways; instead of training the high-level policy to generate subgoals, we select outpost subgoals from the graph. Our effective subgoal selection can avoid the inefficiencies caused by generating inappropriate subgoals. NGTE also accelerates graph expansion through a constrained exploration strategy, leading to an improvement in HRL efficiency.

Our strategy including subgoal selection and exploration shares similarities with the Go-Explore method [8], which groups similarly represented states at the pixel-level into cells, then counts the number of visits to select the least visited state. However, our method selects the most suitable node to achieve the final goal by training Novelty Critic in conjunction with Distance Critic rather than simply relying on count-based method. In addition, our hierarchical structure enables effective and rapid exploration by sampling an

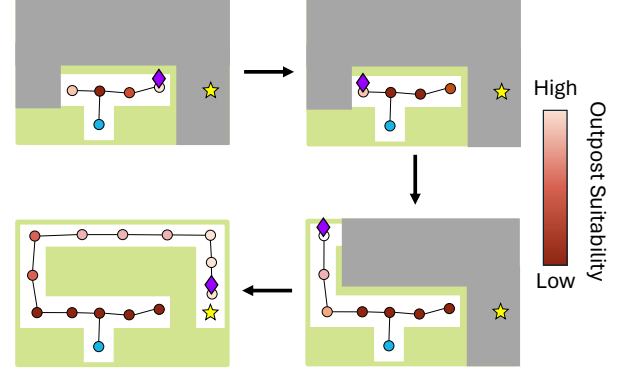


Figure 4: Initially, the agent incorrectly predicted the distance between the node and the final goal, causing it to select the rightmost node as the Outpost Subgoal. As the agent repeatedly experiences the same path nodes over several episodes, the Novelty Values for these nodes decrease. Consequently, the agent naturally selects the node on the left path with the highest suitability (Distance + Novelty) as the Outpost Subgoal. Subsequently, the agent can quickly expand the graph and reach the final goal.

exploration subgoal within constrained boundary and conducting sequence of actions with a trained low-level policy.

3 Preliminaries

3.1 Hierarchical Reinforcement Learning

HRL adopts a multi-level architecture, where a high-level policy operates with a broader perspective on the problem at a lower frequency, providing immediate objectives in the form of subgoals to the lower level. We consider a goal conditioned Markov Decision Process (MDP) [23] represented as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{G}, P, R, \gamma)$, where $\mathcal{S}, \mathcal{A}, \mathcal{G}$ are the state, action, and goal space respectively, P is the transition probability, R is the goal-conditioned reward function and γ is the discount factor. In this MDP setting, agent should maximize cumulative reward by achieving $\mathcal{G}$ as fast as possible.

In sparse reward environments, the agent only receives a reward when it achieves the given goal, leading to a lack of reward signals. To address this problem, many HRL algorithms use hindsight goal relabeling [1] to facilitate reward signals. For a given training set (s_t, a, s_{t+1}, g, r) , goal relabeling involves: 1) replacing the original goal g with an arbitrarily achieved state s_{t+c} in the same episode, and 2) recalculating the reward r to r' using $\text{reward_function}(s_{t+1}, s_{t+c})$. The agent can then receive a reward signal from this new training set $(s_t, a, s_{t+1}, s_{t+c}, r')$. This procedure encourages the agent to consider the direction of the final goal when generating actions.

3.2 Graph and Distance Critic

A graph-based approach of HRL employs a graph $G(N, E)$ comprised of representative visited states as nodes $N \in \mathcal{S}$ and the estimated number of steps between the nodes as edges E . We estimate the number of steps between two states from the binary reward

characteristic of the goal-conditioned problems, following prior works [9, 30]. Concretely, we train parameterized Distance Critic by temporal difference (TD) learning [25] assigning a reward of -1 for each step until the goal is reached. The learned Distance Critic approximates the expected cumulative reward, $Q_{dist} = \sum_{t=0}^n \gamma^t r_t$, where n is the number of steps taken. Then, we can input the state and goal into the Distance Critic to derive the estimated number of steps for the low-level policy π_{low} as follows:

$$n = \log_{\gamma} (1 + (1 - \gamma)Q_{dist}(s, \pi_{low}(s, g)|g)). \quad (1)$$

3.3 Random Network Distillation

In reinforcement learning (RL), complex environments with extensive observation and action spaces require substantial exploration to solve problems. Random Network Distillation (RND) [4] was proposed to address this challenge in RL. RND employs a target network and a predictor network, both randomly initialized, that receive identical inputs randomly sampled from the visited states. The predictor network $\hat{f}(s; \theta)$ aims to forecast the output returned by the frozen target network $f(s; \bar{\theta})$. The discrepancy between the two outputs from the networks is utilized as a loss to train the predictor network, enabling it to accurately predict the output from the target network for states that have been frequently visited.

$$e_{rnd}(s) = \|\hat{f}(s; \theta) - f(s; \bar{\theta})\|_2 \quad (2)$$

$e_{rnd}(s)$ means error between two networks. This error is utilized as an exploration reward, providing substantial error as a reward for states with high uncertainty that have not been frequently visited. Consequently, it encourages the agent to explore states with higher levels of uncertainty.

4 Motivation

Previous HRL algorithms have struggled to generate effective subgoals, as shown in Figure 1, leading to learning inefficiencies. This inefficiency stems from the high-level policy's failure to understand the given environment and final goals due to inadequate exploration of the state space. Without a comprehensive observation of the state space, agents are unable to generate optimal subgoals. Therefore, effective exploration of unexperienced state spaces is required to acquire information about states that can lead to the final goal. For example, if the intended path is blocked by an obstacle, the agent must learn about alternative routes.

Traditional HRL research has allowed for some exploration effectiveness through the self-generation of subgoals and the application of the Hindsight Experience Replay (HER) technique, but the efficiency of this technique decreases as the environmental scale and complexity increase.

Another notable point is that the randomness of the initial and final goal states provided during the learning process significantly affects the difficulty of learning. For instance, in case a), where each episode can start from a random position within the environment, exploration of the entire state space is naturally facilitated [17]. However, this assumes that the environment can provide a random initial state. Conventionally, research has often utilized scenario b), where the initial state is fixed, but the final goal is randomly given at the start of the episode. Because the final goal is randomly provided in the environment, and the agent considers the final goal direction

Algorithm 1 Overview of NGTE

Input: graph $G(N, E)$, final goal g_{final} , π_{low} and Q_{low} for low-level Actor and Critic, Distance Critic Q_{dist} , Novelty Critic V_{nov} , Replay Buffer D

```

1: for  $i = 0 \dots \text{maxEpisodes}$  do
2:   reconstruct  $G(N, E)$  every  $M$  episodes
3:    $s, g_{final} = \text{env.reset}()$ 
4:    $OS = \text{OSS}(G, g_{final}, Q_{dist}, V_{nov}, \gamma)$ 
5:   // OSS(Outpost Subgoal Selection: Algorithm 2)
6:    $Waypoints = \text{Dijkstra}(s, OS, G)$ 
7:   for  $wp$  in  $Waypoints$  do
8:     move to  $wp$  by  $\pi_{low}(s, wp)$ 
9:   end for
10:  while Episode not done do
11:     $g_{exp} = \text{ConstrainedExplorer}(s, RND)$ 
12:    // ConstrainedExplorer: Section 5.5
13:    move to  $g_{exp}$  by  $\pi_{low}(s, g_{exp})$ 
14:  end while
15:  Store all transitions into  $D$ 
16:  Train  $Q_{low}$ ,  $Q_{dist}$ ,  $V_{nov}$ ,  $Rnd$ , and  $\pi_{low}$  using  $D$ 
17: end for
```

in goal-conditioned MDP, these environments promote implicit exploration during training. In case c), where both the initial state and final goal are fixed (often the most challenging state is set as the final goal), no implicit exploration opportunities are provided by the environment, making learning more challenging.

In this research, we introduce the Novelty Critic and use it to rapidly expand the graph by selecting the optimal outpost subgoal on the graph. This approach accelerates state space exploration. Our proposed model not only surpasses existing studies in condition b) but also demonstrates highly effective learning in condition c), where previous studies have nearly failed to learn.

5 Methods

In this section, we describe NGTE, an efficient graph-based HRL algorithm through novelty-based outpost subgoal selection and exploration. NGTE consists of four parts: 1) graph construction, 2) outpost subgoal selection, 3) graph traversal toward the subgoal with planned waypoints, and 4) exploration to expand a graph. Figure 2 and Algorithm 1 introduce all the processes.

5.1 Graph Construction

The graph $G(N, E)$ comprises a set of nodes, N , derived from states visited by the agent during learning, uniformly sampled from the replay buffer. We aim to select nodes sufficiently distant from each other to ensure wide coverage since the nodes serve as potential subgoal candidates. Accordingly, we first pre-sample a set of states and select some that properly represent the visited states using the Farthest Point Sampling (FPS) algorithm with the Distance Critic for calculating step distance. These sampled states perform the state-to-goal space projection function $\psi : \mathcal{S} \rightarrow \mathcal{G}$, resulting in the graph node set N .

We infer the distances of the set of edges E connecting N by learning the Distance Critic. Then, edges are cut off if their predicted

distance is greater than a hyperparameter λ_c . This process removes distant edges that are far away and difficult for the agent to reach from the possible edge candidates. The graph is reconstructed every M episodes, where M is a hyperparameter.

5.2 Outpost Subgoal Selection

When the final goal is given, offering suitable subgoals becomes the most essential technique in HRL algorithms. Previous studies encountered challenges in generating subgoals when the high-level policy was insufficiently trained. Consequently, these generated subgoals often result in misguided directions or are positioned at considerable distances, as illustrated in Figure 1. Determining the frequency of subgoal generation when designing high-level policies also has a significant impact on HRL algorithms [17]. In this paper, instead of training a high-level policy to generate subgoals, we propose an approach to select the most suitable subgoal from the current graph nodes only once per episode.

For the subgoal selection, it is natural to choose the one closest to the final goal among the current graph nodes. However, determining their exact proximity information is challenging when the environment is unknown beforehand. In previous studies, a concept similar to Distance Critic [9, 30] helps to predict the count of (low-level) action steps between two nodes that have been already visited. However, the prediction of distances to unvisited nodes is often unreliable, as depicted in Figure 3(a).

To tackle this problem, we introduce Novelty Critic which learns the expected novelty value from a node to the final goal. As illustrated in Figure 3(b), neighbors to newly visited nodes toward the final goal in the graph acquire higher expected novelty values. Ultimately, as depicted in Figure 3(d), the Distance Critic and Novelty Critic are employed in conjunction, allowing for the selection of subgoals based on values that incorporate both the distance prediction and the expected novelty to the final goal. This effectively enables the selection of subgoals which is the outpost position node towards the final goal.

Algorithm 2 Outpost Subgoal Selection (OSS)

Input: graph $G(N, E)$, final goal g_{final} , γ , Q_{dist} , α , V_{nov}

Output: the most suitable subgoal toward g_{final}

```

1: if  $g_{final}$  located in  $G$  then
2:   subgoal =  $g_{final}$ 
3: else
4:    $q_{dist_i} = Q_{dist}(n_i, \pi_{low}(n_i, g_{final}), g_{final})$   $n_i \in N$ 
5:    $v_{nov_i} = V_{nov}(n_i, g_{final})$   $n_i \in N$ 
6:    $q_i = q_{dist_i} + \alpha v_{nov_i}$ 
7:   distToGoal =  $\{q_1, q_2, \dots, q_{n-1}, q_n\}$ 
8:   subgoal =  $N[\text{argmax}(\text{distToGoal})]$ 
9: end if
10: return subgoal

```

5.3 Learning Novelty Critic

Distance Critic can accurately predict distances between nodes that have already been visited, but it may mistakenly perceive shorter distances to unvisited nodes if there are obstacles such as walls in the way. On the other hand, Novelty Critic V_{nov} learns

the expected novelty value from one state to the given goal using equation 2. Nodes that are visited for the first time or less frequently receive higher novelty rewards from Random Network Distillation (RND), and these rewards are bootstrapped through equation 2 in a temporal difference (TD) learning method

$$V_{nov}(s_t, g) \leftarrow V_{nov}(s_t, g) + \beta(Target - V_{nov}(s_t, g)) \quad (3)$$

where $Target \leftarrow r_{novelty} + \gamma V_{nov}(s_{t+1}, g)$, β is a learning rate, and $s_t, s_{t+1} \in \mathcal{S}$. RND outputs the error shown in Equation (2) and it can be used as an novelty reward called $r_{novelty}$. It is a type of intrinsic reward, which can be described as a self-generated reward that motivates the agent to explore and learn. $r_{novelty}$ is scaled as follows:

$$r_{novelty} = e_{rnd}(s) / EMA(max(e_{rnd}(s)), 0.99) \quad (4)$$

To balance the reward scale between the environment and novelty reward, the error $e_{rnd}(s)$ from RND is normalized by Exponential Moving Average(EMA) of the maximum $e_{rnd}(s)$ in update batch. With the scaled novelty reward $r_{novelty}$, Novelty Critic learns the cumulative $r_{novelty}$ which is obtained by low-level policy from s_t to the arbitrary relabeled subgoal g .

The advantages of the Novelty Critic are as follows: First, the Novelty Critic value can reflect the frequency of state visits. For instance, if an agent repeatedly visits the same path but fails to achieve its goal, the Novelty Critic values of the states along that path will quickly decrease. This plays an important role in preventing the repeated generation of suboptimal subgoals based solely on simple distance predictions using only the Distance Critic. In NGTE, by considering the changing Novelty Critic values based on visitation experience when selecting subgoals, the agent's exploration opportunities are increased, unnecessary time wastage is reduced, and the suboptimality issues mentioned in section 4 can be resolved.

Second, when new states are discovered, the novelty reward (a single RND error) of those states can be bootstrapped to neighboring states, allowing for the generalization of the expected novelty value. For example, if subgoals are selected based solely on the RND error values of individual nodes, as shown in Figure 3(c), even if neighboring states have low RND error values due to frequent visits, a node that is visited for the first time (or infrequently) may have a high RND error. This can lead to unnecessary subgoal selection. In contrast, the Novelty Critic is bootstrapped using Temporal Difference (TD) Learning, resulting in a more generalized representation of novelty values on the graph.

5.4 Low-Level Traversal to Outpost Subgoal

When an episode starts with a given final goal g_{final} , NGTE selects an outpost subgoal OS from the current graph. Using Dijkstra's algorithm, the shortest path from the initial state $\psi(s_0)$ to OS is planned. The graph nodes along the shortest path are referred to as "waypoints" and are denoted as follows $W : (wp_0 = \psi(s_0), wp_1, wp_2, \dots, wp_{k-1}, wp_k = OS)$. The low-level policy observes the state s_t and receives the arbitrary goal-space of state as a temporary subgoal and performs a low-level primitive action $a_t = \pi_{low}(s_t, \psi(s_{t+c}))$. For every low-level step transition, the reward function is defined as: $r = 0$ if $\|\psi(s_{t+1}), \psi(s_{t+c})\|_2 < \lambda_r$, else

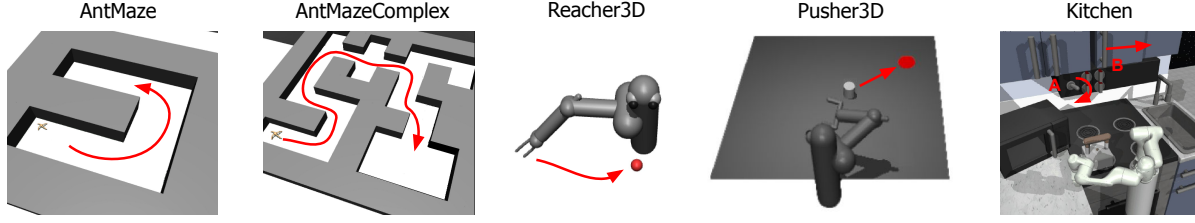


Figure 5: An overview of the various environments is presented herein. The first two pertain to Maze environments where a quadruped locomotion agent, an ant robot, is tasked with reaching challenging points within the maze. The agent is not provided with any map information. The next two environments concern the fetch scenarios, wherein a 7-dof robot arm is assigned tasks to accomplish. The last environment requires 9-dof robot arm to solve 2 different tasks at once.

$r = -1$, where λ_r is a threshold value depending on the environment. The low-level policy is trained by hindsight goal relabeling[1].

5.5 Distance-Constrained Goal Exploration

After reaching the Outpost Subgoal (OS), the exploration phase begins, and the exploration process is repeated for the remaining episodes. In NGTE, no separate policy is learned for exploration. With the help of the Novelty Critic, the agent starts from OS, which is already positioned at the boundary of the graph. Since the areas to visit next are unknown parts of the state space, an exploration policy learned from previous experiences may not always be helpful. Instead, it would require time to learn about each new area. On the other hand, NGTE demonstrates that the graph can be quickly expanded from OS using simple random subgoal-level exploration.

During the exploration phase, NGTE uniformly samples K positions within the average edge length of the graph from the current position. This distance-constrained sampling increases the likelihood that the agent can reach these positions using its low-level policy. For these K states, the RND errors are inferred, and the state with the highest error is selected as the exploration subgoal, denoted as g_{exp} . The g_{exp} is provided as a subgoal for the low-level policy, guiding exploration for C_{exp} steps. After C_{exp} steps, the exploration process restarts from that position.

Hierarchical Reinforcement Learning (HRL) is designed to address long-term, complex problems that require extensive exploration of vast state spaces. Setting exploration subgoals at specific intervals, such as g_{exp} used in NGTE, significantly enhances graph expansion and improves the efficiency of HRL learning compared to single-step exploration. In long-horizon, complex environments targeted by HRL, it is highly efficient to perform actions with temporal consistency. Single-step exploration methods typically restrict the exploration range to an atomic level, making it both time-consuming and difficult to thoroughly investigate the entire state space. [2] In contrast, subgoal-level exploration conducted at specific intervals facilitates the exploration of a broader area in a single attempt. Our approach leverages temporal consistency and ensures the reachability of low-level policies with appropriate distance constraints, thereby optimizing both exploration and learning.

6 Experiments

6.1 Environments

We evaluated NGTE in the MuJoCo simulator [26] which is the physics engine-based robotic control environment. To assess the capacity of HRL in addressing long-horizon and sparse-reward tasks, we selected AntMaze [7], Fetch [5] and Kitchen [10] environments. All the environments are continuous control tasks, with observations provided only as numeric values such as robot joint information, a box position, the goal position, and so on. AntMaze involves a quadruped robot named Ant navigating mazes of various difficulty levels, while Fetch and Kitchen requires controlling a robotic arm manipulation to achieve a given goal. During training, Agents start from fixed initial positions and learn to reach randomly generated goals in AntMaze-Rand and AntMazeComplex-Rand. AntMaze-Fixed, Fetch and Kitchen use a fixed initial position and a fixed goal, while AntMazeComplex-Fixed uses a fixed initial position and 4 fixed challenging positions for goal. Generally, environments that have Random goal and initial position setting can guide the agent to explore state space, which can be helpful for agent [17]. For evaluation, we compared the average scores of multiple different seed models on the same fixed goal. Details of the environments used in the experiments are as follows:

- **AntMaze:** A U-shaped maze with dimensions of 24 x 24 and a success threshold of 5. Absolute x, y coordinates of Ant’s body are given as goal space.
- **AntMazeComplex:** An intricate maze spanning 56 x 56 with a success threshold of 5. During testing, one of four challenging fixed points is randomly sampled and assigned as the goal. Absolute x, y coordinates of Ant’s body are given as goal space.
- **Reacher 3D:** A 7-dof robot must be maneuvered such that its end-effector reaches a designated goal position, with a success threshold of 0.25. Absolute x, y, z coordinates of robot’s end-effector are given as goal space.
- **Pusher 3D:** Operating similarly to the Reacher with a 7-dof robot, this environment requires pushing a block to the target location, with a success threshold of 0.15. Absolute x, y, z coordinates of block are given as goal space.
- **Kitchen:** Multi kitchen tasks environment with 9-dof Franka robot. We specifically address two pivotal tasks: 1) Opening

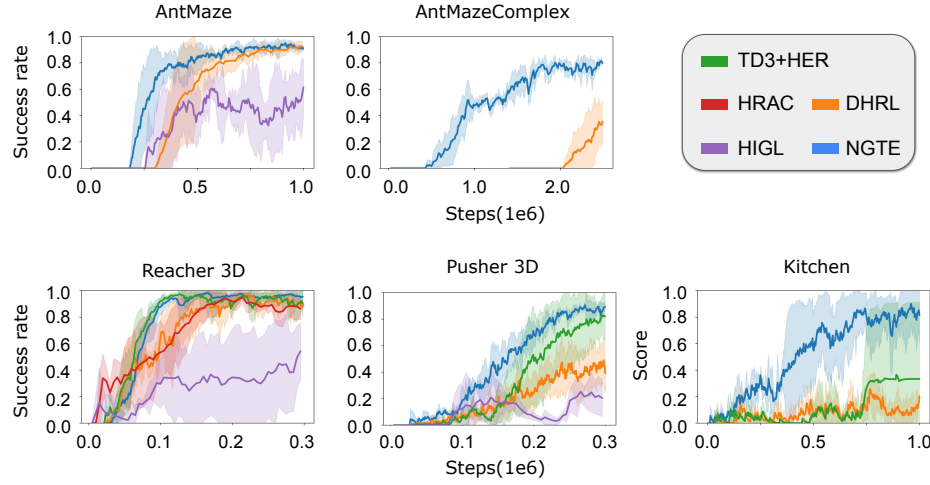


Figure 6: Training graphs of NGTE and baselines for all environments.

	TD3+HER	HRAC	HIGL	DHRL	NGTE
AntMaze-Rand (1M)	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.61 $\pm$ 0.18	0.91 $\pm$ 0.01	0.91 $\pm$ 0.01
AntMaze-Fixed (1M)	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.93 $\pm$ 0.01
AntMazeComplex-Rand (2.5M)	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.37 $\pm$ 0.14	0.78 $\pm$ 0.03
AntMazeComplex-Fixed (2.5M)	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.69 $\pm$ 0.08
Reacher 3D (300K)	0.92 $\pm$ 0.06	0.91 $\pm$ 0.05	0.57 $\pm$ 0.20	0.92 $\pm$ 0.01	0.97 $\pm$ 0.01
Pusher 3D (300K)	0.81 $\pm$ 0.16	0.0 $\pm$ 0.0	0.19 $\pm$ 0.11	0.31 $\pm$ 0.09	0.88 $\pm$ 0.03
Kitchen (1M)	0.33 $\pm$ 0.47	-	-	0.21 $\pm$ 0.09	0.84 $\pm$ 0.18

Table 1: The presented results illustrate the mean success rate over three seeds for each experiment. Notably, NGTE outperforms previous state-of-the-art algorithms across all environments. Especially, in the case of AntMazeComplex, a representative environment with a large observation space, traditional algorithms either fail or low success rate, whereas NGTE demonstrates successful outcomes.

a slide cabinet and 2) Manipulating a light switch rotation, each with a designated success threshold set at 0.3. The agent is rewarded upon successful completion of all tasks. Evaluation is done with the average success ratio of two tasks. Translation of the slide cabinet joint and joint angle value of the light switch are given as goal space. Unlike navigation tasks, the locations of goal-related task objects(cabinet and microwave) are not given. This characteristic forces the agent to explore state space, which makes the task solving harder.

We adopt TD3 algorithm [11] for the low-level policy with the HER method. Farthest Point Sampling is used to construct a graph, and Dijkstra algorithm is used to find the shortest path in the graph. We utilize RND to predict a novelty of a state. Our source code is also uploaded in the supplement and will soon be available on our Github repository.

6.2 Experimental Results

We employed the state-of-the-art algorithms, DHRL and HIGL for graph-based HRL, HRAC for non-graph-based HRL, and TD3+HER for non-hierarchical RL as our baselines. We reproduced the results

using the code provided in the authors’ official GitHub repository for following baselines: DHRL¹, HIGL², HRAC³.

In this section, we conducted all experiments using three different seeds to compare average values. Through our proposed outpost subgoal selection and straightforward exploration techniques, NGTE exhibited superior performance across all environments compared to the baseline methods as shown in Table 1. As NGTE efficiently traverses towards an outpost subgoal for a given final goal and rapidly expands the graph, its excellence is highlighted particularly in larger and more complex challenges.

Figure 6 shows an training graph of each algorithm’s results in various robotic control tasks. NGTE outperformed not only the success rate but convergence time in all environments. Algorithms invisible in graphs are 0 success rate (score) during the training phase.

Among the AntMaze series environments, in the relatively easier AntMaze-Rand setting, Graph-based HRL algorithms were able to

¹https://github.com/jayLEE0301/dhrl_official/tree/main

²<https://github.com/junsu-kim97/HIGL/tree/main>

³<https://github.com/trzhang0116/HRAC>

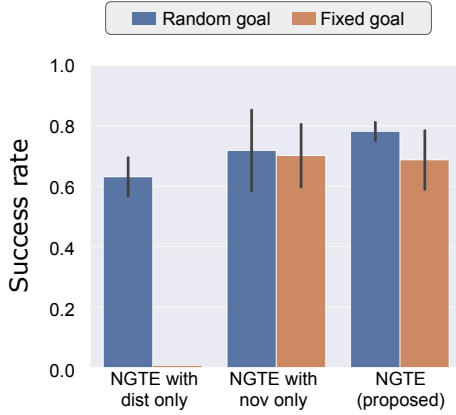


Figure 7: Performance comparison of outpost subgoal selection strategies in AntMazeComplex-Rand and -Fixed: "dist only" and "nov only" refers to selecting an outpost subgoal solely based on Distance Critic or Novelty Critic in NGTE.

reach the goal while TD3+HER and HRAC could not reach the goal within the given timestep budget of 1M. HIGL achieved about 60% success rate of reaching the goals at 1 million steps, while DHRL and NGTE reached a over 90% success rate. In the AntMazeComplex-Rand environment, which has the largest state space to explore, Figure 9 shows DHRL experienced very slow graph expansion at the 2 million steps. On the other hand, NGTE exhibited graph expansion at 1 million steps and, at the 2 million steps, already covered all map. NGTE achieved almost twice of the original SOTA performance at 2.5 million steps. Interestingly, in both AntMaze-Fixed and AntMazeComplex-Fixed settings, only NGTE was able to reach the goal. We conducted additional experiments to analyze the key role of NGTE at Fixed setting in section 6.3.

In the Reacher environment within the Fetch domain, NGTE demonstrated superior performance compared to other baseline algorithms. In both Pusher and Kitchen environments, goal spaces include other objects' information, making it harder for HRL algorithms to decide proper high-level actions. Among HRL algorithms, only NGTE was able to surpassed the performance of TD3+HER.

In Kitchen environment, agent has to carry out all tasks to get a reward signal, which requires agent to explore state space efficiently. TD3+HER showed high variance between different seeds due to unstable exploration, while NGTE showed higher performance with lower variance.

6.3 Fixed Goal Analysis

In this section, we analyze the performance differences of NGTE based on the outpost subgoal selection strategy, depending on whether the goal state is random or fixed during training. Figure 7 shows the performance of variations of NGTE: 'NGTE with dist only', which uses only the Distance Critic, and 'NGTE with nov only', which uses only the Novelty Critic. 'NGTE with dist only' demonstrates sufficient learning in AntMazeComplex-Rand, but shows no learning in AntMazeComplex-Fixed, similar to the results seen in previous studies with HIGH and DHRL that failed to

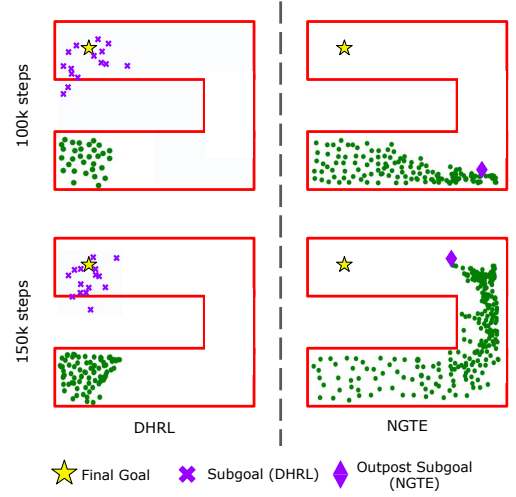


Figure 8: Graph expansion and subgoal positions for previous SOTA algorithm DHRL and NGTE under the fixed-goal condition. DHRL generates a subgoal every 40 steps in an episode. On the other hand, NGTE selects an outpost subgoal once per episode. The fixed goal has a minimal impact on NGTE's performance.

learn. On the other hand, 'NGTE with nov only' shows successful learning in both AntMazeComplex-Rand and AntMazeComplex-Fixed. And NGTE, which uses both distance critic and novelty critic, exhibited the highest performance in all goal settings. Thus, the use of the proposed Novelty Critic not only drives performance improvements in conditions with random goals but also plays a crucial role, especially under fixed goal conditions. Figure 8 represents the graph expansion procedure of DHRL and NGTE in AntMaze. DHRL generates subgoals near the final goal that are unreachable for the low-level policy until the episode ends. In contrast, NGTE's outpost subgoal selection returns highly novel, feasible points from the graph, enabling a successful graph expansion process.

6.4 Exploration Strategies

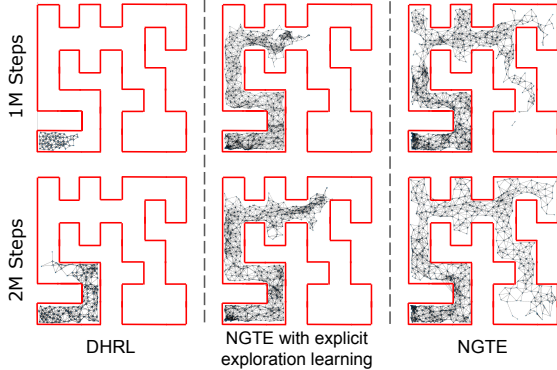
In this section, we study the effect of constrained high-level exploration method compared to a variant of NGTE with Explicit Exploration Learning (EEL). Instead of constrained sampling, the EEL policy is trained to create a exploration subgoal to maximize the RND error between current state and the final goal. EEL consists with critic $Q_{exp}(s_t, a_{exp}, g_{final})$ and actor $\pi_{exp}(s_t, g_{final})$ where g_{final} is the final goal. These networks learn with traditional Actor-Critic training procedure. $Q_{exp}(s_t, a_{exp}, g_{final})$ predicts cumulative RND error between s_t and g_{final} . During the exploration phase, $\pi_{exp}(s_t, g_{final})$ generates a_{exp} as exploration subgoal per every C_{exp} steps for low-level policy.

With three different seed ran for 2M steps, learning high-level policy achieved 0.61 success rate whereas NGTE achieved 0.79 success rate. Figure 9 shows the graph expansion of each algorithm, showing that our constrained high-level exploration method outperforms the policy strategy. We observe that EEL leads to inefficient

	cutoff λ_c	initial rollout episodes	graph reconstruction period M	batch size	max graph nodes
AntMaze	35	10	5	1024	300
AntMazeComplex	40	10	5	1024	500
Reacher	25	200	20	1024	300
Pusher	10	200	10	512	300
Kitchen	10	10	5	512	300

Table 2: Hyperparameters used for environments.

Hyperparameter	Value
π_{low} lr(learning rate)	0.0001
$Q_{low}, Q_{dist}, V_{nov}, RND$ lr	0.001
gamma γ	0.99
Optimizer	Adam
C_{exp}	10
alpha for OSS	0.9
K for g_{exp}	32
Relabeling ratio for Q_{low}	0.8
Relabeling ratio for Q_{dist}, V_{nov}	1.0

Table 3: Hyperparameters used for all environments.**Figure 9: Graph expansion performance comparison in AntMazeComplex: NGTE exhibits significantly faster graph expansion compared to DHRL and NGTE with explicit exploration learning, covering almost the entire map within 2 million steps.**

interactions, generating subgoals within the graph that are often indistinguishable from randomly generated subgoals, whereas our constrained high-level exploration method generally works well. To visualize the graph expansion speed of each algorithm, we plotted the graph coverage of the map at 1M and 2M timestep in Figure 9. It shows that the proposed method contributes to the rapid graph expansion, covering most of the map within 2M steps of environment interaction.

7 Implementation Details

Table 2 and Table 3 shows the details of the hyperparameters that we used for our experiments. We use the same hyperparameters for the network and our main contributions: the Novelty Critic and constrained high-level exploration. We used individual hyperparameter settings for the graph. The hyperparameters in Table 3, except for batch size, significantly affect performance and therefore need to be tuned to fit each environment.

Both Critic and Actor networks are implemented with each three fully connected layers of size 300 and ReLU activation. The output layer of the Actor network uses tanh activation and is multiplied with the action scale parameter. Critic networks output a single value.

8 Conclusion

In this study, we present the Novelty-aware Graph Traversal and Expansion (NGTE) approach. NGTE can enhance HRL by effectively addressing key challenges in large, sparse-reward environments. By selecting Outpost Subgoals at graph boundaries and transitioning into a distance-constrained exploration phase, NGTE ensures efficient graph expansion and reliable final goal achievement. Experimental results in complex environments, such as quadruped robot navigation and robotic arm manipulation, demonstrate NGTE’s superior performance over existing HRL methods. Notably, NGTE is the first to achieve remarkable results in fixed start and fixed goal scenarios, highlighting its robustness and practical applicability. In our future research, we plan to investigate approaches for effectively representing high-dimensional states, such as images, using graph structures.

9 Acknowledgement

This work was partly supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00348376) and the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2024-00438686, Development of software reliability improvement technology through identification of abnormal open sources and automatic application of DevSecOps), (No. 2022-0-01045, Self-directed Multi-Modal Intelligence for solving unknown open domain problems), (No. RS-2022-II220688, AI Platform to Fully Adapt and Reflect Privacy-Policy Changes), and (No. 2019-0-00421, Artificial Intelligence Graduate School Program)

References

- [1] Marcin Andrychowicz, Dwight Crow, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Joshua Tobin, P. Abbeel, and Wojciech Zaremba. 2017. Hindsight Experience Replay. *ArXiv abs/1707.01495* (2017). <https://api.semanticscholar.org/CorpusID:3532908>
- [2] Akhil Bagaria, Jason K. Senthil, and George Dimitri Konidaris. 2021. Skill Discovery for Exploration and Planning using Deep Skill Graphs. In *International Conference on Machine Learning*. <https://api.semanticscholar.org/CorpusID:221094080>
- [3] Andrew G. Barto and Sridhar Mahadevan. 2003. Recent Advances in Hierarchical Reinforcement Learning. *Discrete Event Dynamic Systems* 13 (2003), 41–77. <https://api.semanticscholar.org/CorpusID:386824>
- [4] Yuri Burda, Harrison Edwards, Amos J. Storkey, and Oleg Klimov. 2018. Exploration by Random Network Distillation. *ArXiv abs/1810.12894* (2018). <https://api.semanticscholar.org/CorpusID:53115163>
- [5] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. 2018. Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models. In *Advances in Neural Information Processing Systems*.
- [6] Jonas Degraeve, Federico Felici, Jonas Buchli, Michael Neunert, Brendan Tracey, Francesco Carpanese, Timo Ewalds, Roland Hafner, Abbas Abdolmaleki, Diego de Las Casas, et al. 2022. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature* 602 (2022), 414–419.
- [7] Yan Duan, Xi Chen, Rein Houthoofd, John Schulman, and Pieter Abbeel. 2016. Benchmarking Deep Reinforcement Learning for Continuous Control. In *Proceedings of The 33rd International Conference on Machine Learning*. PMLR, 1329–1338.
- [8] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. 2021. First return, then explore. *Nature* 590 (2021), 580–586.
- [9] Ben Eysenbach, Russ R Salakhutdinov, and Sergey Levine. 2019. Search on the replay buffer: Bridging planning and reinforcement learning. *Advances in Neural Information Processing Systems* 32 (2019).
- [10] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. 2020. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. *arXiv:2004.07219* [cs.LG]
- [11] Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing Function Approximation Error in Actor-Critic Methods. In *International Conference on Machine Learning*. <https://api.semanticscholar.org/CorpusID:3544558>
- [12] Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. 2018. Learning to Walk via Deep Reinforcement Learning. *arXiv:1812.11103* [cs.LG]
- [13] Zhiao Huang, Fangchen Liu, and Hao Su. 2019. Mapping state space using landmarks for universal goal reaching. In *Advances in Neural Information Processing Systems*.
- [14] Max Jaderberg, Wojciech M Czarnecki, Iain Dunning, Luke Marris, Guy Lever, Antonio Garcia Castaneda, Charles Beattie, Neil C Rabinowitz, Ari S Morcos, Avraham Ruderman, et al. 2019. Human-level performance in 3D multiplayer games with population-based reinforcement learning. *Science* 364 (2019), 859–865.
- [15] Junsu Kim, Younggyo Seo, and Jinwoo Shin. 2021. Landmark-guided subgoal generation in hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems*. 28336–28349.
- [16] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. 2016. Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation. In *Advances in Neural Information Processing Systems*.
- [17] Seungjae Lee, Jigang Kim, Inkyu Jang, and H Jin Kim. 2022. DHRL: A Graph-Based Approach for Long-Horizon and Sparse Hierarchical Reinforcement Learning. In *Advances in Neural Information Processing Systems*. 13668–13678.
- [18] Andrew Levy, George Konidaris, Robert Platt, and Kate Saenko. 2017. Learning multi-level hierarchies with hindsight. *arXiv:1712.00948* [cs.AI]
- [19] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, et al. 2021. A graph placement methodology for fast chip design. *Nature* 594 (2021), 207–212.
- [20] Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. 2018. Data-Efficient Hierarchical Reinforcement Learning. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:43920938>
- [21] Ofir Nachum, Haoran Tang, Xingyu Lu, Shixiang Gu, Honglak Lee, and Sergey Levine. 2019. Why does hierarchy (sometimes) work so well in reinforcement learning? *arXiv preprint arXiv:1909.10618* (2019).
- [22] Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. 2021. Hierarchical Reinforcement Learning: A Comprehensive Survey. *Comput. Surveys* 54, 5 (2021), 1–35.
- [23] Martin L. Puterman. 1994. Markov Decision Processes: Discrete Stochastic Dynamic Programming. <https://api.semanticscholar.org/CorpusID:122678161>
- [24] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. 2020. Mastering atari, go, chess and shogi by planning with a learned model. *Nature* 588 (2020), 604–609.
- [25] Richard S. Sutton. 1988. Learning to Predict by the Methods of Temporal Differences. *Machine Learning* 3 (1988), 9–44. <https://api.semanticscholar.org/CorpusID:3349598>
- [26] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 5026–5033.
- [27] Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. 2017. FeUdal Networks for Hierarchical Reinforcement Learning. In *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 3540–3549.
- [28] Vivienne Huiling Wang, Joni Pajarinen, Tinghuai Wang, and Joni-Kristian Kämäräinen. 2023. State-Conditioned Adversarial Subgoal Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 10184–10191.
- [29] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. 2023. DayDreamer: World Models for Physical Robot Learning. In *Proceedings of the 6th Conference on Robot Learning*. PMLR, 2226–2240.
- [30] Lunjun Zhang, Ge Yang, and Bradley C Stadie. 2021. World Model as a Graph: Learning Latent Landmarks for Planning. In *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 12611–12620.
- [31] Tianren Zhang, Shangqi Guo, Tian Tan, Xiaolin Hu, and Feng Chen. 2020. Generating Adjacency-Constrained Subgoals in Hierarchical Reinforcement Learning. In *Advances in Neural Information Processing Systems*. 21579–21590.